

Rigid phase certificates: deciding nine FRACTRAN holdouts, and a decider that explains why

(authorship placeholder — see §9)

August 2026

Abstract

We decide nine open machines of the FRACTRAN Busy Beaver competition BBf(23): each is proved never to halt from its start value $n = 2$. All nine proofs are formalized in Lean 4 without `mathlib` and are `sorry`-free, depending only on Lean’s three core axioms. The proofs are not ad hoc. We isolate a certificate class — *rigid phase certificates* — whose checking is finite, exact and needs no simulation beyond a bounded entry, prove its soundness, and formalize that soundness argument in Lean. The nine machines are exactly the machines our structural detector flags as *rigid* among the 694 refined BBf(23) holdouts; the remaining 675 are digit-consuming, and we explain why no certificate of this class can reach them. A second, equivalence decider — template isomorphism of certificates — resolves the nine into six classes, correcting a plausible-looking but false grouping into three. Along the way the formalization contradicted our own informal proofs on a point of principle: checking that a rival rule is disabled at the endpoints of a run is unsound unless a single guard conjunct is pinned, and we exhibit a machine-checked counterexample.

1 Introduction

FRACTRAN [1] is Conway’s minimal Turing-complete language. A program is a finite ordered list of positive rationals; the state is a positive integer n ; one step replaces n by $n \cdot p/q$ for the first listed fraction p/q with $q \mid n$, and the program halts when no fraction applies. The Busy Beaver variant BBf(k) asks for the longest finite run among programs of description size at most k [2]. Sizes up to 22 are settled; BBf(23) is the live frontier, with 21,233 machines officially undecided as of 10 July 2026 and a refined list of 694 produced by a decider not yet independently reproduced.

Deciding a machine on such a list means proving that a specific, fully explicit program never halts. What makes this hard is that the interesting survivors read their own digits: the next rule to fire depends on arithmetic detail that no bounded-state invariant tracks. The exceptions are machines whose orbit has a *closed form*, and it is a recurring observation that these fall to analysis while the rest do not.

Contributions.

1. **Nine decisions** (§5): the FRACTRAN programs at lines 13649, 14793, 16268, 18226, 18380, 19205, 20255, 20589, 20660 of the official BBf(23) holdout list never halt. [\[proved, Lean-verified\]](#)
2. **A certificate class and a decider** (§3): rigid phase certificates, with a soundness theorem; checking is symbolic in $\{1, n, 2^n\}$ -arithmetic and decides guard positivity for *all* phase indices at once. [\[proved\]](#)

3. **Formalized soundness** (§7): the soundness argument itself is in Lean, and the nine machine theorems are derived as instances of one abstract theorem.
4. **An equivalence decider** (§6): template isomorphism of certificates, decidable by finite search, which resolves the nine into six classes.
5. **A negative result about endpoint checking** (§4): pinning a single guard conjunct is necessary, not merely convenient. [\[proved, Lean-verified\]](#)
6. **A structural census** (§8) locating the boundary of the method: 675 of the 694 refined holdouts are outside it, by their dynamics rather than by our budget. [\[measured\]](#)

Epistemic conventions. Every claim carries a label: [\[proved\]](#) for a hand proof, [\[proved, Lean-verified\]](#) for one machine-checked in Lean 4, [\[measured\]](#) for a measurement over a stated finite domain. We use them because two of the statements below are measurements that could be mistaken for theorems.

2 Preliminaries

Fix the primes $(2, 3, 5, 7, 11)$; every fraction appearing in the machines we treat factors over them. A state is the exponent vector $s = (v_2, v_3, v_5, v_7, v_{11}) \in \mathbb{N}^5$ of n , so that FRACTRAN’s divisibility test becomes a conjunction of coordinate lower bounds and its update becomes vector addition. A *rule* is a pair (guard, delta) with guard a conjunction $\bigwedge_i (s_i \geq \theta_i)$; a *machine* is a priority-ordered list of rules; one step fires the first rule whose guard holds, and the machine halts when none does. We write $s \xrightarrow{m} t$ for “the machine performs m steps from s and is then at t ” (in particular it does not halt on the way), and $\text{NeverHalts}(s)$ for “every step count is completed from s ”.

Lemma 2.1 (normalized halt criterion). *For each machine, the set of halting states is a conjunction of coordinate equalities and at most one disjunction, obtained by negating all five guards and absorbing implications between them. For instance machine 431 halts exactly at $v_2 = v_7 = 0 \wedge (v_5 = 0 \vee v_{11} = 0)$.* [\[proved, Lean-verified\]](#)

The point of Lemma 2.1 is that non-halting becomes a statement about which coordinates stay positive, which is what certificates track.

3 Rigid phase certificates

Definition 3.1 (certificate). Let $\text{EXP} = \{a + bn + c2^n : a, b, c \in \mathbb{Q}\}$. A *rigid phase certificate* for a machine M consists of

- an *entry* (E, i_0) : from the start state, M reaches $B(i_0)$ in E steps;
- a *boundary family* $B(n)$, each coordinate in EXP ;
- *branches*, one per residue class of n modulo 2 (or a single branch), each a list of *stages*: a *run* (rule, count) with count in EXP , or a *block* (K, w, σ) with $K \in \text{EXP}$, w a fixed word of runs, and $\sigma(k)$ a block-start state affine in the block index k with EXP constant terms and integer slopes.

The checker verifies five conditions. **C1 (chain)**: the symbolic composition of the stages equals $B(n + 1)$, and each block’s word-delta equals its start function’s k -slope. Because $\{1, n, 2^n\}$ are linearly independent over $\mathbb{Q}$, state equality is a finite comparison of coefficients.

C2 (guards): the acting rule’s guard holds at the corners of each block rectangle. **C3 (priority)**: for each higher-priority rule, one *fixed* guard conjunct fails at all corners. **C4 (entry)**: by direct simulation. **C5 (coverage)**: the branches cover every $n \geq i_0$.

Two facts make this a decision procedure rather than a search.

Proposition 3.2 (decidable eventual positivity). *For $g \in \text{EXP}$, a threshold t , and an arithmetic progression of indices, “ $g(n) \geq t$ for all admissible $n \geq n_0$ ” is decidable. [proved]*

Proof. If the 2^n -coefficient is negative, or is zero and the linear coefficient is negative, g is eventually below any threshold and the answer is no. Otherwise g is eventually nondecreasing along the progression, with an explicitly computable index N past which its increment is nonnegative; check $n_0 \leq n \leq N$ directly. $\square$

Theorem 3.3 (soundness). *If the checker accepts a certificate for M , then M never halts from its start state. [proved, Lean-verified]*

The proof factors into a purely dynamical part, which is what we formalize, and the book-keeping that C1–C5 discharge.

Theorem 3.4 (phase induction; the certificate contract). *Let $B : \mathbb{N} \rightarrow \mathbb{N}^5$, and suppose $s_0 \xrightarrow{E} B(i_0)$, and that for every $i \geq i_0$ there is $m \geq 1$ with $B(i) \xrightarrow{m} B(i+1)$. Then $\text{NeverHalts}(s_0)$. [proved, Lean-verified]*

Proof. By induction, for every k there are $m \geq k$ and a state t with $s_0 \xrightarrow{m} t$: the base case is the entry, and each phase adds at least one step. A machine that completes arbitrarily many steps completes every step count. $\square$

C4 supplies Theorem 3.4’s entry hypothesis; C1–C3 supply its phase hypothesis for the branch containing i ; C5 ensures every $i \geq i_0$ has one. That the phase hypothesis really follows from C1–C3 is where the geometry enters, and it is the subject of the next section.

4 Corners, and why one conjunct must be pinned

Within a run the state is affine in the run counter; across a block the start state is affine in the block index. Hence every guard value inside a block has the form $a + bk + ct$ with (k, t) ranging over a rectangle.

Theorem 4.1 (corner principle). *If $a + bk + ct \geq 0$ at the four corners of $[0, K] \times [0, L]$, then $a + bk + ct \geq 0$ on the whole rectangle; likewise for $\leq \theta$. [proved, Lean-verified]*

Proof. A linear integer function attains its extremes on an interval at an endpoint; apply this in k along the two extreme rows, then in t for each fixed k . $\square$

Theorem 4.1 licenses C2 immediately: finitely many corner tests replace infinitely many pointwise tests. C3 is subtler, and here our first informal treatment was wrong in a way worth recording.

A rule is *disabled* when *some* conjunct of its guard fails. It is tempting to verify “disabled” at the endpoints of a run and conclude it holds throughout. That inference is invalid: the endpoints may be witnessed by *different* conjuncts.

Proposition 4.2 (endpoint testing of a disjunction is unsound). *There are affine conjunct-values $p + ut$ and $q + vt$ and a run length n such that at $t = 0$ and at $t = n$ some conjunct fails, yet at an intermediate t both hold — so the rule fires in the middle of a run the check has accepted. [proved, Lean-verified]*

line	program	boundary family	phase length
13649	[5/6, 9/35, 8/55, 7/2, 605/7]	$(2^{i+1}-1, 0, 0, 0, i)$	$2^{i+3} - 5$
14793	[63/10, 8/77, 33/2, 5/9, 7/3]	$(2^i+1, 2^i-2, 0, 0, i-1)$	$4 \cdot 2^i$
16268	[9/70, 25/2, 44/15, 7/55, 3/5]	$(0, 0, 2^{i+2}, 0, 2^{i+1}-1)$	$7(2^{i+1}-1) + 4$
18226	[9/35, 5/6, 8/55, 7/2, 605/7]	transport of line 13649	$2^{i+3} - 5$
18380	[7/15, 9/14, 125/77, 2/5, 847/2]	transport of line 13649	$2^{i+3} - 5$
19205	[9/10, 5/21, 343/55, 2/7, 605/2]	transport of line 13649	$2^{i+3} - 5$
20255	[8/15, 147/22, 35/2, 11/49, 3/7]	$(0, 0, 2^k+k, 2^{k+1}-1, 0)$	$5 \cdot 2^k$
20589	[77/30, 88/21, 9/2, 5/11, 7/3]	$(0, 2^{m+1}+2m, 0, 0, 2^m-1)$	$5 \cdot 2^m$
20660	[9/70, 44/15, 25/2, 7/55, 3/5]	$(2^m, 0, 0, 0, 2^m-1)$	$6 \cdot 2^m - 3$

Table 1: The nine decided machines, by line number in the official 21,233-machine BBf(23) holdout list of 10 July 2026. All nine also appear in the refined 694-machine list.

Proof. Take $p + ut = t$ and $q + vt = 2 - t$ with $n = 2$. At $t = 0$ the first value is 0; at $t = 2$ the second is 0; at $t = 1$ both equal 1. $\square$

Accordingly C3 requires a *single fixed* conjunct to fail at all corners, which is sound by Theorem 4.1. The same discipline appears in the Lean development as a design constraint: each firing lemma names the coordinate that disables each higher-priority rival, and a rule typically needs *two* such lemmas because different coordinates do the work in different stages — machine 680’s rule 25/2 is protected by $v_5 = 0$ inside its blocks but by $v_3 = v_7 = 0$ during its opening run. This was the single most common source of error in the formalization, and it is exactly the content of C3.

5 The nine machines

Theorem 5.1. *Each program in Table 1, started at $n = 2$, never halts. [proved, Lean-verified]*

A worked phase. For line 13649, write $W = 2^{i+1} - 1$ and $m = 2^i - 1$. From $B_i = (W, 0, 0, 0, i)$ the guards force

$$f_3^W, \quad (f_4 f_1)^m f_4, \quad f_2, \quad \begin{cases} (f_0^3 f_2)^q f_2^{2q}, & i \text{ even}, 2m = 3q, \\ (f_0^3 f_2)^q f_0^2 f_2^{2q+2}, & i \text{ odd}, 2m = 3q + 2, \end{cases}$$

landing exactly on B_{i+1} in $2^{i+3} - 5$ steps. The case split is forced by $2^{i+1} \bmod 3$; the only nontrivial guard in the entire phase is $v_{11} \geq 1$ at each f_2 , and v_{11} has floor exactly i over the phase. That floor is the machine’s safety margin, and it grows linearly.

Three mechanisms, not one. Lines 13649 and 14793 survive by a *linearly growing fuel* ($v_{11} = i$, resp. $i - 1$). Line 16268 survives by an *exactly preserved invariant*: its boundary satisfies $v_5 = 2(v_{11} + 1)$ and the phase map sends $(v_5, v_{11}) \mapsto (2v_5, 2v_{11} + 1)$, which preserves it. Line 20255 is the only machine of the nine with *no* parity split — its blocks consume nothing three at a time, so no residue condition arises. Lines 20589 and 20660 have *rotated* phase words: the same multiset of rules per block in a different cyclic order, with the rotation depending on the parity.

Margins are not thin. It is worth contrasting these with the one previously decided machine of comparable shape, a BB(6) holdout settled in April 2026 whose halting condition reduces to a 2-adic valuation of $2 \cdot 3^i + i + 5$ staying above a linear bound. That margin is *thin*: it can only be controlled by transcendence methods, and the published proof invokes Baker–Wüstholz.

None of our nine has a thin margin, and no result beyond guard arithmetic and induction is used anywhere below.

6 The equivalence decider

Certificates support a second decision problem. Say two certified machines are *template-isomorphic* if some bijection of rule indices and permutation of the prime axes carries one certificate exactly onto the other. This is decidable by finite search over at most $5! \cdot 5!$ maps, and an isomorphism yields an exact correspondence of orbits from the entry boundaries onward.

Theorem 6.1. *Under template isomorphism the nine machines fall into six classes: {13649, 18226, 18380, 19205} and five singletons. [\[proved\]](#)*

This corrects a natural error. Inspection suggests three families, since 20255, 20589, 20660 each share a fraction multiset with an earlier machine up to reordering. But reordering fractions changes rule *priority*, and priority changes the firing word: those three have genuinely different phase templates, which is why each needed its own proof. The decider settles the question formally rather than by inspection — and the Lean development independently reproduces the same boundary (§7).

7 The Lean development

The formalization is in Lean 4 with **no mathlib**: only core arithmetic, **omega** and kernel **decide**. It comprises 3,262 lines in 9 files, builds from scratch in about four seconds, contains no **sorry**, no **admit**, no **native_decide** and no **axiom** declarations, and each of the nine non-halting theorems depends on exactly Lean’s three core axioms (**propext**, **Classical.choice**, **Quot.sound**); the entry lemmas depend on none.

Fractran.lean defines the semantics and proves the reusable core: a block-iteration lemma and the affine endpoint lemmas. **Decider.lean** proves Theorem 3.4, Theorem 4.1 and Proposition 4.2, and re-derives four of the nine machine theorems through Theorem 3.4, confirming that the certificate contract is exactly what the machine proofs establish.

Two byproducts deserve mention.

Transport as a proof principle. Three of the nine are transports of line 13649. Rather than copy its development, we prove a *firing specification*: six transported firing obligations, stated on the source machine’s role coordinates and embedded by an axis map, from which the entire phase development and induction follow generically. Instantiating it requires discharging only those six facts against the target machine’s own rule list and priority order. The specification demands the top-priority rule fire only at states with $v_7 = 0$, which is precisely what makes the priority swaps in two of the transports harmless — and precisely what the rotated machines cannot satisfy. The formal development thus rediscovers the equivalence boundary of Theorem 6.1 from the other side.

Entries collapse. Our informal case files entered the induction at boundary index 4 after 36, 76 or 78 steps, because their phase templates had been stated only for large indices. The Lean phase lemmas hold for *all* parameter values, and the true entries are far shorter: one step for lines 20255 and 20589, sixteen for 16268, and *zero* for line 20660, whose start state (1, 0, 0, 0, 0) already *is* its index-0 boundary. The old counts survive as ground-truth cross-checks, which is how we confirmed the re-indexing was faithful rather than a slip.

8 Scope: what rigid certificates cannot do

A detector for the certificate class costs little: accelerate the machine exactly, segment its run at occurrences of a marker rule, and classify the resulting per-phase series. We ran it over both holdout lists.

list	rigid (GEO)	polynomial	non-rigid
refined, 694 machines	9	10	675
official, 21,233 machines	3,253	6,797	11,183

Table 2: Structural census of the BBf(23) holdouts. [\[measured\]](#)

The reading is that rigidity is the *decidable margin* of the holdout set: about 15% of the official list is rigid, and after the community’s newest deciders have run, exactly 9 rigid machines survive — the nine decided here. What remains is overwhelmingly non-rigid: 675 of 694. Those machines choose their next rule from arithmetic detail that grows with the state, so no boundary family in EXP — indeed no family with a bounded description — tracks them, and the certificate class is empty for them by construction rather than by budget. This is the same phenomenon that makes generalized Collatz problems hard, and we make no claim to address it.

We also note what did *not* appear: among the nine rigid survivors, none has a thin, Baker-type margin. If machines of that kind exist on these lists, they are not in the rigid class we can detect, and the natural place to look is the BB(6) Turing-machine list, where extracting an arithmetic reduction is itself the hard step.

9 Data and reproducibility

All code, certificates, logs and the Lean development accompany this paper. The checker runs the nine certificates in about a second; a falsifier that systematically perturbs boundaries, counts, rule indices and block coefficients produced 540 corrupted certificates, all rejected, none accepted. Every machine proof is additionally cross-checked against direct big-integer FRACTRAN simulation.

One faithfulness boundary, stated plainly. The Lean development formalizes the exponent-vector machine, and its agreement with divisibility FRACTRAN — that dividing by q is subtracting q ’s exponent vector — is checked computationally rather than proved, since a formal proof needs unique factorization. Closing that gap is straightforward with `mathlib` and is the obvious next step for anyone wishing to remove the last informal link.

Provenance. Holdout lists are those published by the BBf project: the official size-23 list of 10 July 2026 (21,233 machines) and the refined list of 25 July 2026 (694 machines, produced by a decider not yet independently reproduced; we do not rely on it — all nine machines are on the official list).

References

- [1] J. H. Conway. Fractran: a simple universal programming language for arithmetic. In *Open Problems in Communication and Computation*, Springer, 1987, 4–26.
- [2] The BBFractran project. Holdout lists and enumeration conventions for BBf(k). <https://github.com/int-y1/BBFractran>
- [3] The bbchallenge Collaboration. <https://wiki.bbchallenge.org>

- [4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. *CADE-28*, 2021.